PROJECT BRIEF // CLOSEST NUMBER

CO-OPERATIVE CODING

The brief for the team on stage: Delta and the Volunteer. What we build, who owns what, the contract we share, t phases and issues, and the rules for every pull request.



DATE
WED 18 NOV

TIME
16:30-19:30

TEAM
DELTA + VOLUNTEER

REPO
github.com/Delta-43/cooperative-coding-workshop

HOW TO USE THIS BRIEF

Before the day
Volunteer: do section 00. It takes about 30 minutes.

On the day
Sections 05 and 06 are the plan we follow on stage.

Keep open
Appendix A has every command we run.

CONTENTS

00 Get ready (Volunteer)
01 The game
02 Team and roles
03 Architecture
04 The contract
05 Phases and issues
06 Rules for every PR
07 Skills at named moments
08 Design outline
09 Safety and teardown
10 If something breaks
A Commands

SECTION 00 · VOLUNTEER

GET READY

Do this at home before the day, so the workshop time goes into building. If a step fails, copy the exact error and send it to Delta.

01 **GitHub.** Accept Delta's invitation to `Delta-43/cooperative-coding-workshop`. You get write access to this one repository only.

02 **Tools.** Install Git, Node.js 22 or newer, and the GitHub CLI (`gh`). Check with `git --version`, `node --version` and `gh --version`.

03 **A token for one repo.** On GitHub, create a fine-grained personal access token. Repository access: only `cooperative-coding-workshop`. Permissions: Contents, Issues and Pull requests, read and write. Expiry: 7 days. Then run `gh auth login --with-token` and paste it.

04 **Claude Code.** Install it, run `claude` and log in. Keep the permission prompts on.

05 **Clone and run the frontend.**

```
git clone https://github.com/Delta-43/cooperative-coding-workshop.git
cd cooperative-coding-workshop/frontend
npm ci
npm run dev        # open http://localhost:5173/#/host
```

06 **Try the flow once.** Create a branch, change one word in `frontend/README.md`, open a draft PR with `gh pr create --draft`, then close it.

DONE WHEN

- [ ] The host screen opens on your laptop and shows a room code.
- [ ] Your phone can join that room with the QR code.
- [ ] You opened and closed a draft PR from the terminal.

Never paste a token, password or `.env` file into an agent. You never need backend or Cloudflare credentials: the frontend talks to the live Worker over the internet.

SECTION 01

THE GAME

Closest Number is a live quiz for a room. The host screen is on the projector. Players scan a QR code, pick a nickname and answer number questions on their phones. The closest guess wins the round.

STEP	HOST SCREEN	PHONE
`Lobby`	QR code, room code, how many have joined	Room code and nickname, then "You're in"
`Question`	The question, the unit, how many have answered	The question and a number field. An answer can change until the reveal
`Answer`	Every guess beside a bar; Space fills the bar to the answer; then the top 3	The answer, your guess, how far off, your place and points
`Final`	The winner and the top 5	Your place, your points and your best guess

SCORING

The closest guess scores 3 points, then 2, then 1. "Closest" means the smallest relative error: the distance divided by the answer, because the answers range from 16 to over a million. **Equal distance: the earlier answer ranks higher.** Changing an answer restarts your time. Equal final scores: the lower total answer time ranks higher.

SECTION 02

TEAM AND ROLES

Delta: coordinator and backend
Owns `backend/` and the Worker. Merges the contract. Signs off every phase. Narrates for the room.

Volunteer: frontend
Owns `frontend/`: host screen, phone screens, QR join. Reviews Delta's PRs. Never gets backend credentials.

Each of us drives our own Claude Code in our own clone. One agent per person. The contract is the only file we both change, and only in phase 1. The plan lives in issues and milestones, so the humans hold it, not the agents.

SECTION 03

ARCHITECTURE



PATH	WHAT IT HOLDS	OWNER
contract/	`messages.md`, `schema.json` and an example of every message	Both
backend/	The Worker, the room logic, scoring and tests	Delta
frontend/	React, TypeScript and Vite. The design layer is `src/styles/game.css`	Volunteer
AGENTS.md	Short and hand-written: how to run, test and name things. Both agents read it	Both
DECISIONS.md	One line per decision, committed with the PR that made it	Both

The backend address is a setting (`VITE_BACKEND_URL`), so the frontend can point at the live Worker, a local one or a test one.

SECTION 04

THE CONTRACT

Every message is JSON over one WebSocket per player: `/api/rooms/KQWZ/ws` . The host adds `?host=<token>` . The full text is `contract/messages.md` .

MESSAGE	FROM	WHAT IT DOES
join	Player	`{ name, playerId? }` . A saved `playerId` rejoins after a dropped connection
answer	Player	`{ value }` . Allowed in the question phase. A new answer replaces the old one and restarts the time
start · reveal · next	Host	Move the game: lobby → question → reveal → next question … → over
kick	Host	`{ playerId }` . Removes a player and closes their connection
state	Server	The whole game for this client, after every change. `reveal` stays `null` until the reveal: the answer never leaks early
error	Server	`{ code, message }` : `name_taken` , `room_full` (30), `wrong_phase` , `not_allowed` and more

SECTION 05

PHASES AND ISSUES

Each phase is a milestone with one **"Phase N sign-off"** issue that only Delta closes, after he checks every "done when". Each task issue has a "done when", an exact verify command and a "do not touch" line. The PR shows the verify output. Run each verify command in the folder the issue owns.

PHASE 1 · CONTRACT (BOTH, 10 MIN)

ISSUE	DONE WHEN	VERIFY	DO NOT TOUCH
C1 Merge the contract on the skeleton	Every example validates. Both of us approved the PR	`cd backend && npm test`	`backend/src`, `frontend/src`

PHASE 2 · BUILD IN PARALLEL (55 MIN)

ISSUE	DONE WHEN	VERIFY	DO NOT TOUCH
B1 Rooms and join codes	A host creates a room; players join with the code; a taken name is refused	`npm test`	`frontend/`
B2 Answer intake	Answers are accepted only in the question phase; a new answer replaces the old one	`npm test`	`frontend/`
B3 Closest-number scoring	3-2-1 points by relative error; equal error goes to the earlier answer	`npm test`	`frontend/`
F1 Join screen	Code and nickname; wrong code and taken name show a clear error	`npm run lint` `npm run build`	`backend/`, `contract/`
F2 Answer screen	Number field with the unit; the send button stays above the phone keyboard; "change answer" works	`npm run lint` `npm run build`	`backend/`, `contract/`
F3 Host leaderboard	Top 3 after the reveal, top 5 at the end; readable from the back row (16 px minimum on a 960 × 540 frame)	`npm run lint` `npm run build`	`backend/`, `contract/`

PHASE 3 · INTEGRATE AND PLAY (15 MIN)

ISSUE	DONE WHEN	VERIFY
I1 Merge both sides and deploy	Pages shows the new frontend; the room plays a full round	`node test/e2e.mjs <live URL>`

Small-change lane: a typo or copy fix needs only an issue and a PR, with no phase sign-off.

SECTION 06

RULES FOR EVERY PR

BRANCH	One branch per issue, named after it: `b3-scoring`, `f2-answer-screen`. One agent works on it.
SIZE	Small enough to read in five minutes. If it grows, split the issue.
CI	Must pass before merge: backend tests, contract examples, frontend lint and build. The branch rule blocks the merge button until it does.
REVIEW	The other person approves. Agents may comment; a human approves. Delta signs off the phase.
EXPLAIN	Before merging, the author explains the PR with the agent closed: what changed, and what breaks if it is wrong.
FRESH EYES	One read-only subagent reviews the diff against the issue in a clean context, and reports. It does not edit.
DECISIONS	Any decision goes into `DECISIONS.md` as one line, in the same PR.

SECTION 07

SKILLS AT NAMED MOMENTS

Each skill does one job at one moment and reports. It does not rewrite the code.

MOMENT	WHO	SKILL	ITS JOB
Contract PR, before merge	Both	**`API Contract Review`**	Find gaps and ambiguities in the messages
Backend PRs	Delta	**`Clean Code`**	Naming, size and clarity of the room logic
Join and answer screens	Volunteer	**`Impeccable`**	Critique and polish against the design outline
Surfaces and icons	Volunteer	**`better-ui`**	Hit areas, alignment, icon details
Leaderboard motion	Volunteer	**`improve-animations`**	One motion moment, eased, with reduced motion
Phone screens	Volunteer	**`Apple HIG`**	Tap targets, spacing and type on a phone

SECTION 08 · VOLUNTEER

DESIGN OUTLINE

Comic panels on a night pond: a navy field, square panels with thick ink edges, halftone dots and hard offset shadows with no blur. One idea per screen. The projector screens must read from the back row.

FIVE COLOURS, ONE JOB EACH

#011627	#FDFFFC	#2EC4B6	#FF9F1C	#C8102E
field + ink	text + paper	actions	the winner	stickers

TYPE

Anton for titles and big numbers: first word filled, second outlined. **Switzer** for questions and body text. **JetBrains Mono** for every number, code and key.

SHAPES

Every corner is square. A painted panel has three parts: a 3 px navy edge, a halftone fill and a hard shadow (teal or white, because navy would vanish on navy).

BUTTONS

Main: white paper, navy capitals, teal shadow. Second: teal paint, white shadow. Hover lifts 2 px; pressing pushes into the shadow.

RULES FOR THE ROOM

- [ ] Nothing to read under 16 px on the 960 × 540 host frame.
- [ ] Built for 30 players: counts and short name lists, not one row each.
- [ ] No names of slow or wrong players on the projector.
- [ ] Orange only for the winner, once per screen.
- [ ] The phone's send button sits under the field, above the keyboard.
- [ ] Space does the main action on every host screen.

The finished version is in `frontend/src/styles/game.css`. On the day you rebuild a new design from this outline and the contract.

SECTION 09

SAFETY AND TEARDOWN

- [ ] No secrets in the repo or in any prompt. Players use nicknames only.
- [ ] Agents act only on issues we write. Comments from the room are restated by a person first.
- [ ] Permission prompts stay on. No "skip permissions" mode.
- [ ] Nothing reaches `main` without a PR, green CI and a human approval.

AFTER THE WORKSHOP

- [ ] The Volunteer deletes their token; Delta removes their access.
- [ ] Delete any test Worker; rotate every token used on stage.
- [ ] Check the repo's activity log for anything unexpected.
- [ ] The Volunteer runs `gh auth status` to confirm they are logged out.

SECTION 10

IF SOMETHING BREAKS

Wi-Fi blocks WebSockets	Use a phone hotspot. Last resort: the recording
Live rebuild goes wrong	Switch to the checkpoint branch of the finished build
An agent is slow	Use the pre-built frontend branch, and say so

APPENDIX A

COMMANDS

```
# backend (Delta)
cd backend
npm ci
npm test
npm run dev               # Worker on :8787
node test/e2e.mjs         # a full round,
local
npm run deploy            # Delta only

# frontend (Volunteer)
cd frontend
npm ci
npm run dev               # :5173
npm run lint
npm run build

# the flow
gh issue list
git switch -c f2-answer-screen
gh pr create --fill
gh pr checks --watch
gh pr review --approve    # the other
person
```

HOST KEYS

Space	Start, reveal, fill the bar, next. Twice on the final screen: new game
Backspace	Step back on the host screen
Esc	Players menu: remove a player